Week 7 APP DEV-2

2024 Aug04 Quiz-2

1. [Source Q11; MCQ]

Suppose you are organizing a felicitation program for the students with exemplary performance in the BS program. You have the scorecard of all the students. The administration has decided to shortlist the students based on the following 2 criteria:

1. The student must have an "S" in their MAD II project
2. The overall CGPA of the student must be greater than 8

Fill in the ***code1*** & ***code2,*** which can be used in Vuex Store to update the "awardees" state variable with the objects of those students who satisfy the above-mentioned criteria. The objects to be inserted in the "awardees" state variable must only have the student ID and a random token generated in the "send_task" action function.

```
const store = new Vuex.Store({
    state : {
        bs_program : [
            {
                student_id : 'stu1001',
                mad2_grade : 'S',
                cgpa : 8.35
            },
            {
                student_id : 'stu1002',
                mad2_grade : 'A',
                cgpa : 8.78
            },
            {
                student_id : 'stu1003',
                mad2_grade : 'S',
                cgpa : 9.1
            },
            {
                student_id : 'stu1004',
                mad2_grade : 'S',
                cgpa : 7.44
            },
        ],
```

```
        awardees : []
    },

    mutations : {
        update_final(state, payload) {

            for (student of state.bs_program)
                code2
        }
    },
    actions : {
        send_task : function (context) {
            const token = get_token(); // generates a random token
            code1
        }
    }
})
```

(a)

```
code1: this.$store.commit("update_final", {"min" : 8, "token" : token});
code2: if (student.mad2_grade != "S" && student.cgpa > 8) {
            const obj = {};
            obj.student_id = student.student_id;
            obj.token = payload.token
            state.awardees.push(student)
        }
```

(b)

```
code1: context.commit("update_final", 8, token);
code2: if (mad2_grade != "S" && cgpa > 8){
            awardees.push(company)
        }
```

(c)

```
code1: this.$store.commit("update_final", 8, token);
code2: if (mad2_grade != "S" && cgpa > 8){
            awardees.push(company)
        }
```

(d)

```
code1: context.commit("update_final", {“min” : 8, “token” : token});
code2: if (student.mad2_grade != “S” && student.cgpa > 8) {
          const obj = {};
          obj.student_id = student.student_id;
          obj.token = payload.token
          state.awardees.push(student)
       }
```

[Source ans: d]

2. [Source Q13; MSQ]

Which of the following statement(s) is/are incorrect regarding Vuex?

(a) A Vuex store provides a single source of truth that can drive the application.

(b) It provides a variable named “this.$vuexstore” to allow components to accessthe store data.

(c) A component cannot have its own local state, if the application uses Vuex.

(d) The mutations and actions are some constructs of a Vuex store.

[Source ans: b, c]

2024 Oct27 Quiz-1

3. [Source Q3; MCQ]

Which of the following is an example of ephemeral state in a web application?

(a) A user's authentication token stored in local storage

(b) A form input value that changes as the user types

(c) The list of favorite items saved in a user's profile

(d) A shopping cart saved between sessions

[Source ans: b]

4. [Source Q16; MSQ]

Which of the following scenario(s) represent ephemeral state(s) in a web application?

(a) The user's current scroll position on a webpage.

(b) The user's authentication token, saved in localStorage.

(c) The temporary state of a dropdown menu.

(d) All of these

[Source ans: a, c]

2024 Dec01 Quiz-2

5. [Source Q9; MCQ]

In Vuex, what is the purpose of mutations?

(a) To commit asynchronous operations

(b) To directly mutate the state

(c) To get the state

(d) To define computed properties

[Source ans: b]

6. [Source Q11; MCQ]

Consider the below Vuex implementation.

```
const store = new Vuex.Store({
  state: {
    token: sessionStorage.getItem('authToken') || null
  },
  mutations: {
    setToken(state, token) {
      state.token = token;
      sessionStorage.setItem('authToken', token);
    },
    clearToken(state) {
      state.token = null;
      sessionStorage.removeItem('authToken');
    }
  },
  actions: {
    login({ commit }, token) {
      commit('setToken', token);
    },
    logout({ commit }) {
      commit('clearToken');
    }
  }
});
```

After the user logs in with a token and refreshes the page, what will happen to the Vuex state and the session storage?

(a) The session storage will reset, but the Vuex state will still contain the“authToken”.

(b) Both the Vuex state and the sessionStorage will retain the token after the pagerefresh.

(c) The Vuex state will load the token from sessionStorage after the page refresh.

(d) The token will be lost from both the Vuex state and sessionStorage after thepage refresh.

[Source ans: c]

7. [Source Q15; MSQ]

Which of the following statement(s) accurately describe the relationship between system state and application state?

(a) System state often includes user-specific data that changes frequently.

(b) Application state can depend on system state, but not the other way around.

(c) System state is more general and global, while application state is user-specific.

(d) Changes in system state often require a complete page reload to take effect.

[Source ans: b, c]

2024 Dec22 ET

8. [Source Q10; MCQ]

What's the issue with this Vuex mutation? Assuming this mutation is part of vuex store.

```
mutations: {
  updateUser(state, userData) {
    state.user = userData
    state.lastUpdated = Date.now()
    if (userData.role === 'admin') {
      fetchAdminData().then(data => {
        state.adminData = data
      })
    }
  }
}
```

(a) Multiple state changes in one mutation

(b) Async operation in mutation

(c) Direct state mutation

(d) Both Multiple state changes in one mutation and Async operation in mutation

[Source ans: b]

9. [Source Q25; MCQ]

Given the following Vuex store setup.

```
const store = new Vuex.Store({
  state: {
      counter: 0
  },
  mutations: {
      increment(state) {
          state.counter++;
      }
  },
  actions: {
      async incrementAsync({ commit }) {
          await new Promise(resolve => setTimeout(resolve, 1000));
          commit('increment');
      }
  }
});
```

Assuming the store is correctly binded with a Vue app, what will be the correct behavior when the following code is executed?

```
this.$store.dispatch('incrementAsync');
```

(a) The increment mutation will be called immediately after the dispatch.

(b) The state will be updated after the asynchronous operation completes.

(c) The incrementAsync action will be executed synchronously, and increment willbe committed before the promise resolves.

(d) The action will be skipped since mutations cannot be called inside actions.

[Source ans: b]

10. [Source Q28; MCQ]

Consider the following HTML, which includes Vue 2 and Vuex CDN links.

```
<div id="app">
        <p v-if="userData">User: {{ userData.name }}</p>
        <p v-else>No user data available</p>
    </div>

    <script>
        const store = new Vuex.Store({
            state: {
                userData: null
            },
            mutations: {
                setUserData(state, data) {
                    state.userData = data;
                }
            },
            actions: {
                fetchUserData({ commit }) {
                    return fetch('https://httpstat.us/500')
                        .then(response => {
                            if (!response.ok) throw new Error('Network error');
                            return response.json();
                        })
                        .then(data => {
                            commit('setUserData', data);
                        },
                            error => {
                                commit('setUserData', { name: 'Luke' })
                            })
                        .catch(() => {
                            commit('setUserData', null);
                        });
                }
            }
```

```
});


new Vue({
    el: '#app',
    store,
    created() {
        this.$store.dispatch('fetchUserData');
    },

    computed: {
        userData() {
            return this.$store.state.userData;
        }
    }
});

</script>
```

What will be rendered on the browser, if the fetch request gets back 500 status code?

(a) The userData in the store is set to the data from the API.

(b) User: null

(c) The error will be logged, Nothing will be displayed

(d) User: Luke

[Source ans: d]

2025 Mar16 Quiz-2

11. [Source Q9; MCQ]

Consider the below Vuex setup.

```
const store = new Vuex.Store({
  state: {
    cart: [
      { id: 1, name: 'Item 1', price: 100 },
      { id: 2, name: 'Item 2', price: 200 }
    ]
  },
  getters: {
    totalPrice(state) {
      return state.cart.reduce((total, item) => total + item.price, 0);
    }
  },
  mutations: {
    addToCart(state, item) {
      state.cart.push(item);
    }
  },
  actions: {
    addItemToCart({ commit }, item) {
      commit('addToCart', item);
    }
  }
});
```

What will happen if the user dispatches the "addItemToCart" action to add a new item to the cart?

(a) The total price will remain the same until the page is refreshed.

(b) The total price will not reflect the new item because the getter is not reactive.

(c) The total price will be updated immediately in the Vuex store.

(d) A mutation error will occur because the cart array is not updated.

[Source ans: c]

2025 Apr13 ET

12. [Source Q10; MCQ]

Consider the below Vuex store configuration:

```
const store = new Vuex.Store({
  state: {
    items: [],
  },
  mutations: {
    addItem(state, item) {
      state.items.push(item);
    },
  },
  actions: {
    addItemAsync({ commit }, item) {
      setTimeout(() => {
        commit('addItem', item);
      }, 1000);
    },
  },
});
```

If the addItemAsync action is dispatched with the item 'Vue.js', what will be the state of items after 2 seconds?

(a) []

(b) ['Vue.js', 'Vue.js']

(c) ['Vue.js']

(d) [object]

[Source ans: b]

13. [Source Q20; MCQ]

Consider the below Vuex setup.

```
const store = new Vuex.Store({
  state: {
    value: 0
  },
  mutations: {
    setValue(state, payload) {
      state.value = payload;
    }
  },
  actions: {
    async updateValue({ commit }) {
      const newValue = await fetchData();
      commit('setValue', newValue);
    }
  }
});

function fetchData() {
  return new Promise(resolve => {
    setTimeout(() => resolve(42), 2000);
  });
}

store.dispatch('updateValue');
console.log(store.state.value);
```

Consider the below Vuex setup.

What is logged to the console immediately after dispatching the action?

(a) 0

(b) 42

(c) Undefined behavior

(d) Error: Action did not resolve

[Source ans: a]

14. [Source Q29; SA]

Consider the below Vuex store configuration.

```
const store = new Vuex.Store({
  state: {
    count: 0,
  },
  mutations: {
    increment(state) {
      state.count++;
    },
  },
});
```

And the below Vue component:

```
<template>
  <div>
    <button @click="increment">Increment</button>
    <p>Count: {{ count }}</p>
  </div>
</template>

<script>
import { mapState, mapMutations } from 'vuex';

export default {
  computed: {
    mapState(['count']),
  },
  methods: {
    mapMutations(['increment']),
  },
};
</script>
```

What will be displayed in the placeholder "{{ count }}" on the browser after clicking the "Increment" button 3 times?

[Source ans: 3]

2025 Aug03 Quiz-2

15. [Source Q11; MSQ]

Consider the following Vue 2 code with Vuex 3 used via CDN, assume the Vuex store is correctly injected into the Vue instance.

Filename: store.js

```
export default store = new Vuex.Store({
  state: {
    score: 10,
  },
  mutations: {
    increaseScore(state) {
      state.score += 5;
    },
  },
  actions: {
    increaseScoreAsync({ commit }) {
      setTimeout(() => {
        commit("increaseScore");
      }, 1000);
    },
  },
});
```

Filename: score-display.js

```
export default Vue.component('score-display', {
  template: `
    <div>
      <p>Current Score: {{ score }}</p>
      <button @click="boostNow">Boost Now</button>
      <button @click="boostLater">Boost After 1s</button>
    </div>
  `,
  computed: {
    score() {
      // CODE 1
    }
```

```
  },
  methods: {
    boostNow() {
      // CODE 2
    },
    boostLater() {
      // CODE 3
    }
  }
});
```

Which of the following correctly fills in CODE 1, CODE 2, and CODE 3 for the score to be displayed and for the `boostNow()` and `boostLater()` methods to work correctly?

(a)

```
CODE 1: return this.$store.state.score;
```

(b)

```
CODE 2: this.$store.commit('increaseScore');
```

(c)

```
CODE 2: this.increaseScore();
```

(d)

```
CODE 3: this.$store.dispatch('increaseScoreAsync');
```

(e)

```
CODE 3: this.$store.increaseScoreAsync();
```

[Source ans: a, c, d]

16. [Source Q15; MCQ]

Consider the following Vuex store setup for a user management system:

```
const store = new Vuex.Store({
  state: {
    users: [
      { id: 1, name: 'Alice', status: 'active', role: 'admin' },
      { id: 2, name: 'Bob', status: 'inactive', role: 'user' },
      { id: 3, name: 'Charlie', status: 'active', role: 'user' }
    ]
  },
  getters: {
    activeUsers(state) {
      return state.users.filter(user => user.status === 'active');
    },
    activeUserCount(state, getters) {
      return getters.activeUsers.length;
    },
    adminUsers(state) {
      return state.users.filter(user => user.role === 'admin');
    }
  },
  mutations: {
    updateUserStatus(state, { userId, status }) {
      const user = state.users.find(u => u.id === userId);
      if (user) {
        user.status = status;
      }
    },
```

```
    addUser(state, user) {
      state.users.push({ ...user, id: Date.now() });
    }
  },
  actions: {
    toggleUserStatus({ commit, state }, userId) {
      const user = state.users.find(u => u.id === userId);
      if (user) {

        const newStatus = user.status === 'active' ? 'inactive' : 'active';
        commit('updateUserStatus', { userId, status: newStatus });
      }
    },

    async createUser({ commit }, userData) {
      await new Promise(resolve => setTimeout(resolve, 100));
      commit('addUser', userData);
    }
  }
});
```

What will happen when the following sequence of operations is executed?

```
store.dispatch('toggleUserStatus', 1);  // Toggle Alice's status
store.dispatch('createUser', { name: 'David', status: 'active', role:
'user' });
```

(a) The `activeUserCount` will become 1 immediately after the first dispatch, and remain 1 after the second dispatch completes.

(b) The `activeUserCount` will become 1 after the first dispatch, and become 2 after the second dispatch completes.

(c) The `activeUserCount` will remain 2 after the first dispatch, and become 3 after the second dispatch completes.

(d) The `activeUserCount` will become 3 immediately after both dispatches, since both operations run synchronously.

[Source ans: b]

2025 Aug31 ET

17. [Source Q19; MSQ]

Which of the following are true about mutations and actions in Vuex?

(a) Actions can be asynchronous

(b) Only actions can call APIs through fetch call

(c) Mutations should modify state directly

(d) Mutations can be asynchronous

[Source ans: a, b, c]

2025 Dec18 ET

18. [Source Q10; MCQ]

Which of the following is true about Vuex state?

(a) It should be modified only through mutations

(b) It should be modified only through actions

(c) It cannot contain nested objects

(d) It should be modified by getters

[Source ans: a]

2026 Mar15 Quiz-1

19. [Source Q9; MCQ]

Which of the following best describes UI State (Ephemeral State) in frontend applications?

(a) It includes all data stored permanently in the database and is shared across users

(b) It represents the complete system data such as users, products, and transactions

(c) It refers to short-lived interface elements like loading indicators or selected tabs

(d) It handles complex application logic and long-term session management

[Source ans: c]

20. [Source Q10; MCQ]

Given that HTTP is stateless, which approach is commonly used to manage application state between the client and the server?

(a) Storing all state permanently in frontend memory across sessions

(b) Allowing the frontend to handle complex business logic and data storage

(c) Client or server maintaining state and explicitly exchanging it through requests

(d) Eliminating state entirely from web applications

[Source ans: c]

2026 Apr06 Quiz-2

21. [Source Q2; MCQ]

Consider the following Vue.js code.

```
const store = new Vuex.Store({
  state: { todos: [] },
  mutations: {
    addTodo(state, todo) { state.todos.push(todo) }
  },
  actions: {
    async addTodoAsync({ commit }, todo) {
      commit('addTodo', { text: todo, done: false });
      return 'done';
    }
  }
});

store.dispatch('addTodoAsync', 'Learn Vuex')
  .then(msg => console.log(msg));

console.log(store.state.todos.length);
```

What will be printed on the console first?

(a) 0

(b) done

(c) 1

(d) Error

[Source ans: c]

22. [Source Q6; MCQ]

Consider the following Vuex store setup for a user management system:

```
const store = new Vuex.Store({
  state: {
    users: [
      { id: 1, name: 'Alice', status: 'active', role: 'admin' },
      { id: 2, name: 'Bob', status: 'inactive', role: 'user' },
      { id: 3, name: 'Charlie', status: 'active', role: 'user' }
    ]
  },
  getters: {
    activeUsers(state) {
      return state.users.filter(user => user.status === 'active');
    },
    activeUserCount(state, getters) {
      return getters.activeUsers.length;
    },
    adminUsers(state) {
      return state.users.filter(user => user.role === 'admin');
    }
  },
  mutations: {
    updateUserStatus(state, { userId, status }) {
      const user = state.users.find(u => u.id === userId);
      if (user) {
        user.status = status;
      }
    }
  }
});
```

Suppose the following mutation is committed:store.commit("updateUserStatus", { userId: 2, status: "active" });. Which of the following statements is TRUE after this mutation is executed?

(a) `activeUserCount` will return 3

(b) `adminUsers` will return two users

(c) `activeUsers` will include only Alice and Charlie

(d) The mutation will not affect any getter output

[Source ans: a]

2026 May06 ET

23. [Source Q8; MCQ]

Consider the below Vuex store configuration:

```
const store = new Vuex.Store({
 state: {
        items: ['Angular', 'Vue'],
 },
 mutations: {
   addItem(state, item) {
     state.items.push(item);
   },
 },

 actions: {
   addItemAsync({ commit }, item) {
     setTimeout(() => {
       commit('addItem', item);
     }, 500);
   },
 },
});
```

If the addItemAsync action is dispatched with the item 'React', what will be the state of items after 2 seconds?

(a) ['Angular', 'Vue']

(b) ['Angular', 'Vue', 'React']

(c) ['React']

(d) ['Angular']

[Source ans: b]

24. [Source Q23; MCQ]

A student is going on a trip and wants to check whether all essential items are packed. The application uses Vue 2 and Vuex to manage the list of packed items.The following Vue application is incomplete. Two lines of code are missing and are marked as Line X and Line Y.

```
<div id="app">
  <h3>Trip Essentials</h3>

  <ul>
    <li v-for="item in packedItems" :key="item">
      {{ item }}
    </li>
  </ul>

  <button @click="packItem('Passport')">Pack Passport</button>
</div>

<script>
const store = new Vuex.Store({
  state: {
    packed: []
  },
  mutations: {
    ADD_ITEM(state, item) {
      state.packed.push(item);
    }
  }
});



new Vue({
  el: "#app",
  store,
  computed: {
    /* Line X */
  },
  methods: {
    packItem(item) {
      /* Line Y */
    }
  }
});
</script>
```

Which of the following options correctly fills Line X and Line Y so that the application works as expected?

(a)

```
// Line X
packedItems() {
  return this.store.state.packed;
}

// Line Y
this.store.commit("ADD_ITEM", item);
```

(b)

```
// Line X
packedItems() {
  return this.$store.state.packed;
}

// Line Y
this.$store.commit("ADD_ITEM", item);
```

(c)

```
// Line X
packedItems() {
  return this.$store.packed;
}

// Line Y
this.commit("ADD_ITEM", item);
```

(d)

```
// Line X
packedItems: this.$store.state.packed

// Line Y
this.$store.dispatch("ADD_ITEM", item);
```

[Source ans: b]

2026 Aug14 Quiz-2

25. [Source Q7; MCQ]

Consider the following Vuex store snippets. Which of the following demonstrates the correct usage of a mutation as per its definition?

(a)

```
mutations: {
  ADD_TO_CART(state, product) {
    state.cart.push(product)
  }
}
```

(b)

```
mutations: {
  ADD_TO_CART(state, product) {
    setTimeout(() => {
      state.cart.push(product)
    }, 500)
  }
}
```

(c)

```
mutations: {
  ADD_TO_CART(state, product) {
    fetch('/api/add').then(() => {
      state.cart.push(product)
    })
  }
}
```

(d)

```
mutations: {
  ADD_TO_CART(state, product) {
    Promise.resolve().then(() => {
      state.cart.push(product)
    })
  }
}
```

[Source ans: a]

26. [Source Q12; MCQ]

Fill in Code1, inside a mutation handler, which can be used to reduce the price of each of the products in the state by a certain discount. app.js:

```
const store = new Vuex.Store({
    state: {
      discount:10,
      products:[
        {name:'apple',
        price:120
        },
        {
          name:'banana',
          price:60
        }
      ]
    },
    mutations: {
        reducePrice:state=>{
            // Code1

        }
      }
  })
```

(a)
```
this.$store.state.products.forEach(product => { product.price -= (state.discount)/100*product.price; })
```

(b)
```
state.products.forEach(product => { product.price -= (state.discount)/100*product.price;})
```

(c)
```
commit.products.forEach(product => { product.price -= (state.discount)/100*product.price;})
```

(d)
```
dispatch.products.forEach(product => { product.price -= (state.discount)/100*product.price;})
```

[Source ans: b]

27. [Source Q13; MCQ]

Consider the following Vuex store and component code. store.js

```
const store = new Vuex.Store({
  state: {
    cart: []
  },
  mutations: {
    ADD_TO_CART(state, product) {
      state.cart.push(product)
    }
  },
  actions: {
    addToCart({ commit }, product) {
      setTimeout(() => {
        commit('ADD_TO_CART', product)
      }, 500)
    }
  }
})
```

Component.vue

```
methods: {
  addItem_A(product) {
    this.$store.commit('ADD_TO_CART', product)
  },
  addItem_B(product) {
    this.$store.dispatch('addToCart', product)
  },
  addItem_C(product) {
    this.$store.state.cart.push(product)
  }
}
```

After triggering all three methods (`addItem_A` , `addItem_B` and `addItem_C`) one by one, which of the following statements is correct?

(a) All three methods will update the cart and all changes will be visible in Vue DevTools mutation history.

(b) Only `addItem_A` and `addItem_B` will appear in Vue DevTools mutation history, but `addItem_B` will appear with a delay.

(c) Only `addItem_B` will update the cart because mutations cannot be called directly from components.

(d) `addItem_C` will throw an error because Vuex prevents direct state modification.

[Source ans: b]

2026 Sep13 ET

28. [Source Q6; MCQ]

Consider the following Vuex store:

```
const store = new Vuex.Store({
  state: {
    temperature: 30
  },
  getters: {
    weatherStatus(state) {
      return state.temperature >= 35 ? "Hot" : "Normal"
    }
  }
})
```

The `weatherStatus` getter is used in a Vue component as shown below:

```
computed: {
  weatherStatus() {
    return this.$store.getters.weatherStatus
  }
}
```

Now suppose the Vuex getter `weatherStatus` is removed, while the temperature state continues to be updated through Vuex mutations. Which of the following implementations correctly recreates `weatherStatus` in the component so that it automatically reacts whenever the Vuex `temperature` changes?

(a)

```
computed: {
  weatherStatus() {
    return this.$store.state.temperature >= 35
      ? "Hot"
      : "Normal"
  }
}
```

(b)

```
watch: {
  weatherStatus() {
    return this.$store.state.temperature >= 35
      ? "Hot"
      : "Normal"
  }
}
```

(c)

```
methods: {
  weatherStatus() {
    this.$store.state.temperature >= 35
      ? "Hot"
      : "Normal"
  }
}
```

(d)

```
watch: {
  weatherStatus() {
    return this.$store.state.temperature >= 35 }
}
```

[Source ans: a]